LARGE LANGUAGE MODELS: PRODUCTION AND DEPLOYMENT

LLM Inference & Inference Latency

From User Prompt to Generated Response

LECTURE 5 | DURATION · 60 MIN | FROM TRAINING TO SERVING

TOPICS COVERED

- TTFT
- TPOT / ITL
- End-to-End Latency
- Autoregression
- Prefill vs Decode
- KV Cache
- Batching
- Throughput
- p50 / p95 / p99
- Streaming



WHERE ARE WE IN THE COURSE?

From Training to Serving

LECTURE 1
Foundations

→

LECTURE 2
Pre-training, Prompting, RAG, Fine-Tuning

→

LECTURE 3
Fine-Tuning Process and Types

→

YOU ARE HERE

LECTURE 5
LLM Inference + Latency

→

LECTURE 4
PEFT → LoRA → QLoRA

→

LECTURE 6
Compression, Quantization, Pruning, Distillation

> Lecture 4 asked: How do we train efficiently?
>
> **Lecture 5 asks: How do we respond efficiently?**

TODAY'S ROUTE

Inference basics → Latency metrics & throughput → Production reality: KV cache, queueing, tail latency

SECTION 01 / 03

How Generation Works

From a trained model to one generated token — what inference is, how tokens are produced, and the two phases of generation.

IN THIS SECTION

- What Is Inference
- Training vs Inference
- Inference Pipeline
- Autoregression
- Prefill vs Decode

SECTION 01 · FOUNDATIONS

What Is LLM Inference?

> Inference is the process of using a trained model to generate predictions or responses for new inputs.

1. User Prompt
2. Tokenizer
3. Token IDs
4. LLM Forward Computation
5. Next-Token Scores
6. Token Selection
7. Generated Token

then repeat until completion

Training vs Inference

TRAINING	INFERENCE
Forward pass	Forward pass
Loss · backprop · gradients	None of these
Optimizer + parameter updates	Weights remain fixed
Memory requirement	High → usually lower
Purpose	Change what the model knows → Use what the model learned

TRAINING → parameters change

INFERENCE → parameters normally remain fixed

> Inference uses what the model learned; training changes what the model knows.

SECTION 01 · SYSTEM VIEW

The Complete Inference Pipeline



! Inference latency is not only GPU computation time.

What the complete application may also include:

Network · Queueing · Tokenization · Model computation · Decoding · Detokenization · Response transmission

SECTION 01 · CORE MECHANIC

Generation Is Autoregressive

PROMPT “Large language models are”

↓

MODEL PREDICTS **powerful**

↓

NEW CONTEXT “Large language models are powerful”

↓

PREDICTS AGAIN **tools**

↓

RESULT “Large language models are powerful tools”

Context → Predict next token → Append token → Predict again → Repeat

Output tokens are usually generated sequentially — this sequential dependency is one reason LLM inference differs from ordinary single-pass classification.

CLASSIFICATION

Input → One Forward Computation → Class

one pass, one result

GENERATION

Prompt
↓
Token 1
↓
Token 2
↓
Token 3
↓
…
↓
Token N

each generated token becomes part of the next context

“If I ask the model to generate 500 tokens instead of 50, should I expect the same response time?” **No.**

SECTION 01 · TWO PHASES

ONE OF THE MOST IMPORTANT IDEAS IN THIS LECTURE

Prefill & Decode

1 PREFILL

Process the complete input prompt.

`Prompt Tokens [1, 2, 3, 4, …, N]` → Transformer

↓

Internal representations | KV Cache

parallel work across all N prompt tokens

2 DECODE

Generate output tokens one at a time.

Token 1

↓

Token 2

↓

Token 3

↓

…

one token per sequential step

> Prefill processes the prompt. Decode generates the response.

SECTION 01 · SCALING BEHAVIOR

Prompt Length vs Output Length

PREFILL SIDE · PROMPT LENGTH

1000 tokens vs **4000 tokens**

longer prompt usually requires more prefill work

1000 prompt tokens → Transformer computation →
First output token becomes possible

`Prompt length ↑ → Prefill work ↑ → TTFT often ↑`

TTFT — defined in the next section

DECODE SIDE · OUTPUT LENGTH

Token 1 → Token 2 → Token 3 → …

`Output length ↑ → More decode steps ↑ → End-to-end latency ↑`

Request A — generate 20 tokens

Request B — generate 1000 tokens

even with identical prompt and TTFT, B needs many more decode iterations

maximum output-token configuration directly influences worst-case request duration and serving cost

Long prompt → mainly affects **PREFILL**

Long response → mainly affects **DECODE duration**

SECTION 02 / 03

Measuring Latency

One latency number is never enough — TTFT, TPOT, end-to-end time, and the throughput we gain by batching.

SECTION AGENDA

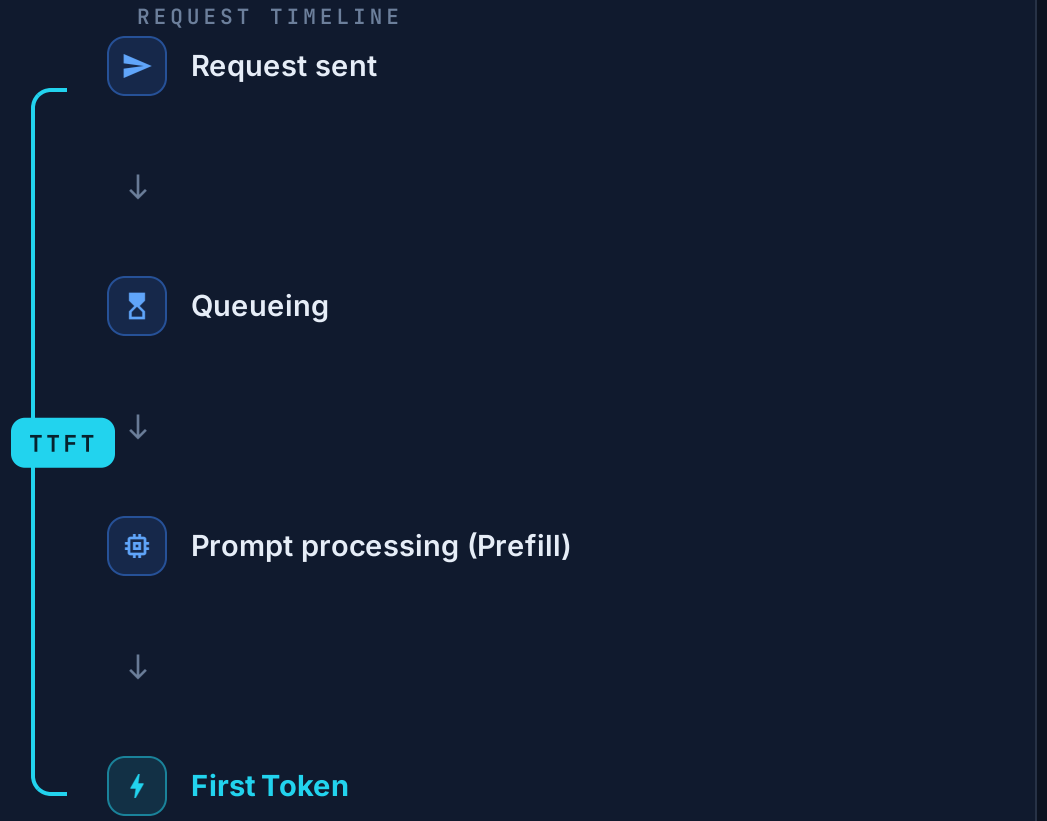
- TTFT
- TPOT / ITL
- E2E Latency
- Latency vs Throughput
- Batching

SECTION 02 · METRIC 1

Time to First Token (TTFT)

Latency is the time required for a request or operation to complete — **but for LLMs, one latency number is not enough.**

- **TTFT** — Time to First Token
- **TPOT / ITL** — Time per Output Token / Inter-Token Latency
- **E2E** — End-to-End Request Time



WORKED EXAMPLE

Request at:	10:00:00.000
First token at:	10:00:00.850

TTFT = 850 ms

Low TTFT makes the system feel responsive — **especially when tokens are streamed.**

SECTION 02 · METRIC 2

Time per Output Token / Inter-Token Latency

TPOT / ITL

TPOT ≈

25 ms / token

STREAMED OUTPUT TOKENS



FORMULA

1 / 0.025 = 40

tokens per second — equivalent approximate generation speed

DEFINITION

TPOT / ITL = average delay between consecutive generated tokens, measured after the first token appears

⏩ Lower TPOT → faster token generation

SECTION 02 · METRIC 3

End-to-End Latency — and What Users Feel

CLASSROOM EXAMPLE

E2E latency = total time from request submission until the complete response is produced.

TTFT	= 0.8 s
Output	= 50 tokens
Generation	≈ 40 tokens/s
First token	≈ 0.8 s
Remaining 49 tokens	≈ 49 / 40 ≈ 1.23 s

Total ≈ 2.03 s

SYSTEM A

TTFT 0.3 s

Total 5 s

SYSTEM B

TTFT 2 s

Total 4 s

Which feels faster to the user?

Often **A feels more responsive initially** — output begins quickly.

Perceived responsiveness and total completion time are not identical.

SECTION 02 · TWO GOALS

Latency vs Throughput

LATENCY

Time for one request.

`Request → Response · 2 seconds`

THROUGHPUT

Amount of work completed per unit time.

`Requests / second` · `Tokens / second`

Low latency and high throughput are related but different optimization goals.

Be careful with “tokens per second”

PER-REQUEST GENERATION SPEED

`user request → 40 output tokens/s`

SERVER AGGREGATE THROUGHPUT

`100 concurrent requests → 3000 total tokens/s`

These are not the same metric.

Always specify what the tokens-per-second measurement represents.

SECTION 02 · BATCHING

Batching & the Latency–Throughput Trade-Off

Batching means processing **multiple requests or tokens together.**



BENEFITS

- better hardware utilization
- higher aggregate throughput

DRAWBACKS

- waiting / queueing
- different request lengths complicate scheduling

BATCH SIZE = 1 — **Request A → GPU alone:** low waiting time, but GPU may be underutilized

BATCH SIZE = 16 — **A B C … P → GPU together:** throughput ↑, GPU utilization ↑, but individual requests may wait longer

STATIC BATCHING PROBLEM

Request A → 10 output tokens	finishes early
Request B → 200 output tokens	keeps running
Request C → 50 output tokens	finishes later

tightly grouped → resources may not be used optimally — **this motivates more flexible serving strategies**

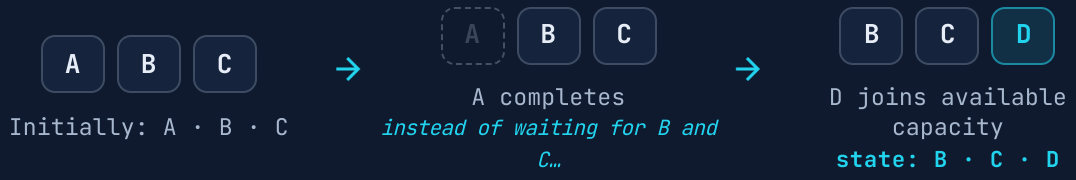
! Optimizing server throughput can sometimes increase per-request latency.

SECTION 02 · SMARTER SERVING

Continuous Batching & Concurrency

CONTINUOUS BATCHING

Time →



Continuous batching dynamically adds new requests as capacity becomes available.

- better GPU utilization
- improved throughput
- more efficient production serving

detailed server implementation is not necessary in this lecture

CONCURRENCY

Concurrency = number of requests being handled at the same time.

- 1 — **1 user** → simple
- 100 — **100 users** → scheduling + queueing
- 10K — **10,000 users** → production serving problem

Increasing concurrency may: Throughput ↑

but also: Queueing latency ↑ · Memory pressure ↑

SECTION 03 / 03

Production Reality

KV cache, model size, hardware, queueing and tail latency — serving is a system, not a forward pass.

SECTION AGENDA

- KV Cache
- Bottlenecks
- Queueing & Cold Start
- p50 / p95 / p99
- Benchmarking

SECTION 03 · MEMORY

! CRITICAL CONCEPT

What Is the KV Cache — and Why It Helps

During autoregressive decoding, previous tokens' attention information would otherwise need repeated computation. The model stores previously computed **Keys + Values** for earlier tokens — hence **KV Cache**.

✕ WITHOUT CACHE

Generate token **101** → recompute information for tokens **1…100**

Generate token **102** → recompute **1…101**

Generate token **103** → recompute **1…102**

⚠ Wasteful.

✓ WITH KV CACHE

Previous K/V — already stored + New token

↓

Compute only new K/V

The model avoids recomputing previous key/value projections during every decode step.

> “KV cache trades memory for faster decoding.”

SECTION 03 · MEMORY COST

The KV Cache Is Not Free

KV memory ∝ Layers × Sequence Length × KV Heads × Head Dimension × Precision × Batch Size

and there are both K and V components stored per layer.

Longer context → larger KV cache

More concurrent users → larger total KV memory

And context is not free either

Prompt A — 20 tokens vs Prompt B — 8,000 tokens

Which one reaches the first output token faster? **Usually A, all else equal.**

Longer prompts increase:

- tokenization work
- prefill computation
- attention work
- KV-cache allocation
- potentially TTFT

Context is useful — but context is not free.

SECTION 03 · WHAT SLOWS INFERENCE

Model Size, Hardware & Bottlenecks

01 MODEL SIZE

0.5B — less computation / memory → 7B — more → 70B — much more

larger models typically require more memory, memory bandwidth, compute, and communication if distributed

Larger does not automatically mean proportionally slower — hardware and serving strategy matter.

02 HARDWARE

CPU vs GPU vs Multiple GPUs — same model — dramatically different performance

compute capability · GPU memory · memory bandwidth · inter-GPU communication · CPU/GPU transfer · accelerator architecture

03 BOTTLENECK TYPE

COMPUTE-BOUND
performance limited mainly by arithmetic operations

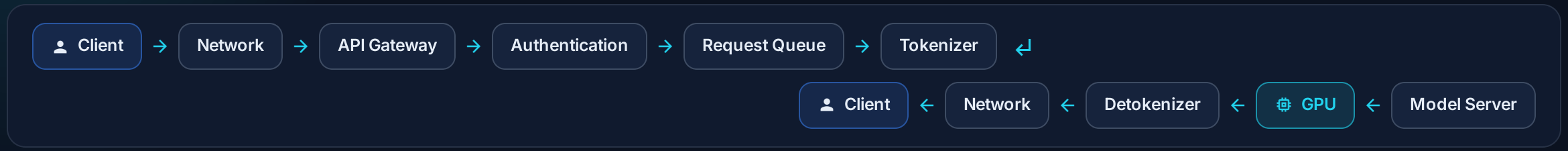
MEMORY-BANDWIDTH-BOUND
performance limited by moving model/data through memory

LLM inference can exhibit different bottlenecks depending on: prefill vs decode · batch size · model architecture · hardware · precision.

> “A faster GPU is useful only if the bottleneck is where the GPU is faster.”

SECTION 03 · THE FULL PATH

Production Latency Is More Than Model Latency



E2E latency ≠ GPU kernel time only.

QUEUEING

Imagine all GPUs are busy.

Request → QUEUE → wait... → GPU becomes available

```
model execution: 500 ms
+ queue wait: 2 s
---
= 2.5 s user latency
```

Capacity planning is part of latency optimization.

COLD START vs WARM

COLD START start server · load weights · allocate GPU memory · initialize kernels/runtime · warm caches — **can take substantially longer**

WARM REQUEST model already loaded and serving

Never compare cold-start latency with steady-state latency without stating the difference.

SECTION 03 · MEASURING RIGHT

Tail Latency & Fair Benchmarking

% PERCENTILES

p50 = 1.2 s | 50% of requests finish within 1.2 s

p95 = 3.8 s | 95% within 3.8 s

p99 = 8.5 s | 99% within 8.5 s

☹ **Users suffering at p99 do not care that the average looks good.**

WHY P99 CAN BECOME BAD

queue congestion · very long prompts · very long generations · traffic spikes · memory pressure · hardware contention · network variability · cold/warm differences

tail latency often reveals problems that averages hide

BENCHMARKING

Record at least:

- ✓ model
- ✓ hardware
- ✓ precision
- ✓ prompt length
- ✓ output length
- ✓ batch / concurrency
- ✓ TTFT
- ✓ TPOT / ITL
- ✓ E2E latency
- ✓ throughput
- ✓ memory usage

(+ preferably p50 / p95 / p99)

ⓘ **"A latency number without workload context is nearly meaningless."**

— Latency = 1.2 s: for what prompt length? how many output tokens? what GPU? what concurrency?

FAIR COMPARISON

same hardware · same workload · same request distribution · same concurrency · same output constraints

Model A: 100-token prompt, 20-token response vs Model B: 4000-token prompt, 500-token response — then declaring A faster: **obviously unfair.**

SECTION 03 · CASE STUDY

Streaming & a Real Request

STREAMING ~~Complete response → send to user~~ → Token 1 → user · Token 2 → user · Token 3 → user …

perceived latency improves

streaming does not necessarily reduce total computation — it mainly improves responsiveness.

UNIVERSITY HELPDESK

"My examination form is not opening and today is the final date."

Request → Authentication → Optional RAG retrieval → Prompt construction → LLM inference → Streaming answer

TIME TO FIRST TOKEN

Retrieval 150 ms | Queueing 100 ms | Prefill 300 ms → TTFT total 550 ms

GENERATION

Decode 40 tokens/s → 80-token answer ≈ 2 s generation

Where would you optimize first? — encourages systems thinking.

When inference is slow, ask: **Where is time being spent?**

Queue? | Tokenization? | Prefill? | Decode? | Network? | Retrieval/tools? →

Then optimize the actual bottleneck.

> **"Never optimize latency by guessing. Measure first."**

SECTION 03 · CLASSROOM EXERCISE

Two Numerical Challenges

1 CHALLENGE 1 — TOTAL LATENCY

GIVEN

- TTFT = 0.7 s
- Average output speed = 50 tokens/s
- Required output = 101 tokens

SOLVE

First token: 0.7 s → Remaining: 100 / 50 = 2.0 s

$$0.7 + 2.0 = 2.7 \text{ s}$$

✓ **≈ 2.7 seconds** — assuming reasonably steady decode speed and ignoring other complications.

2 CHALLENGE 2 — THROUGHPUT

GIVEN

- 20 simultaneous requests
- Each produces 100 output tokens
- All complete in ≈ 5 seconds

COMPUTE

$$(20 \times 100) / 5 = 400 \text{ output tokens/s}$$

? "Does that mean every user individually received 400 tokens/s?"

No. the difference between **per-request speed** and **aggregate throughput**.

SECTION 03 · CALIBRATION

Misconceptions — and What to Try First

COMMON MISCONCEPTIONS

Inference latency means only GPU computation.	FALSE
Higher throughput always means lower latency.	FALSE
Long prompts only affect output quality.	FALSE
KV cache reduces memory. *it uses memory to reduce repeated computation*	FALSE
Streaming makes the model compute fewer tokens.	FALSE
Average latency alone is enough.	FALSE
If inference works on a GPU, performance is automatically acceptable.	FALSE

OPTIMIZE BEFORE COMPRESSION

Without changing the model itself, investigate:

- → shorten unnecessary prompts
- → limit excessive output length
- → use streaming
- → reduce queueing
- → improve batching / scheduling
- → use KV cache correctly
- → select appropriate hardware
- → avoid unnecessary preprocessing
- → warm the model
- → control concurrency
- → choose appropriate model size

"But we can go further — by changing the representation or architecture of the model itself." **That is Lecture 6.**

LECTURE 5 · WRAP-UP

Summary & The One Sentence to Remember

Inference
use a trained model to generate responses

Prefill
process the input prompt

Decode
generate output tokens sequentially

TTFT
how long until the user sees the first token

TPOT / ITL
how quickly subsequent tokens appear

Throughput
how much total work the system completes

KV Cache
trades memory for faster decoding

Batching
improves hardware utilization but can affect latency

DO NOT CONFUSE:

- Fine-tuning changes behavior
- Quantization changes numeric precision
- KV cache stores attention states
- Batching groups work
- Streaming returns tokens incrementally
- Compression reduces model/storage/compute requirements

> “Fast LLM serving is not simply about making one forward pass faster; it is about managing prefill, decoding, memory, scheduling, queueing and concurrency as one production system.”

What if the model itself is simply too large — too much memory, too many bytes through the GPU?

→ LECTURE 6 → Compression → Quantization → Pruning → Knowledge Distillation

Lecture 5 taught us where inference time comes from. Lecture 6 will teach us how to make the model itself lighter.